

CryptoQuant Terminal

Phase 1: Complete System Flow Audit

Full System Map from Data Input to Final Output

Date: 2026-06-04

Version: 1.0 - Read Only (No Modifications)

Total Files Analyzed: 316 | API Routes: 104 | Prisma Models: 47

1. Executive Summary

This document presents the complete system flow map for CryptoQuant Terminal, built from an exhaustive analysis of all 316 source files, 104 API routes, 47 Prisma models, 80+ service/library modules, and the architecture document (ARCHITECTURE_FINAL.md v1.0). The goal of this phase is purely observational: to map how information flows through the entire system from data input to final output, identify all modules, dependencies, integrations, data transformations, and decision points, and document the current state without making any modifications.

The analysis reveals a system with a well-defined layered architecture but significant integration gaps between layers. While individual modules are functional, the critical pipeline that should connect validation modules (Monte Carlo, Walk-Forward, Backtest) through a Strategy Decision Engine (SDE) to Capital Allocation and Paper Trading execution remains partially disconnected. The system currently produces information rather than decisions, and several modules exist as isolated islands generating metrics that nobody synthesizes into actionable outputs.

1.1 Key Metrics

Metric	Value	Notes
Source Files (src/)	316	Including services, APIs, components, stores
API Routes	104+	Across 39 route groups
Prisma Models	47	1512 lines of schema
Service Modules	80+	Across 7 subdirectories
UI Components	50+ shadcn/ui + 35 dashboard	Single-page app with 19 tabs
Zustand Stores	2	crypto-store + deep-analysis-store
Data Source Clients	9	DexScreener, CoinGecko, DexPaprika, Etherscan, Binance, Dune, Footprint, SQD, OHLCV Pipeline
Allocation Methods	16	5 active per architecture, 6 deprecated, 3 delayed
Auth	DISABLED	Hardcoded demo user everywhere

2. Layered Architecture

The system follows a 5-layer architecture pattern. Each layer depends only on the layers below it, forming a strict dependency hierarchy. Data flows from external APIs at the bottom, through processing and analysis layers, to the UI at the top.

Layer	Components	Dependencies
-------	------------	--------------

Layer 5: UI	Single-page app (19 tabs), 2 Zustand stores, WebSocket/SSE real-time feeds, React Query for API calls	Depends on: API Routes
Layer 4: API Routes	104+ REST endpoints across 39 groups, all use runtime=nodejs + dynamic=force-dynamic, zero auth middleware	Depends on: Services + DB
Layer 3: Execution	paper-trading-engine, autonomous-execution-engine, trade-execution-engine, sync-shared	Depends on: Brain + Strategy + Risk
Layer 2: Brain + Strategy + Risk	brain-orchestrator, brain-cycle-engine, SDE, TDE, strategy-evolution, kill-switch, capital-allocation, monte-carlo, operability-score	Depends on: Data Sources + Backtesting
Layer 1: Data Sources + Backtesting	DexScreener, CoinGecko, DexPaprika, Etherscan, Binance, OHLCV Pipeline, backtesting-engine, walk-forward, feedback-loop	Depends on: Infrastructure
Layer 0: Infrastructure	db (Prisma), unified-cache, utils, format, validations, ws-bridge, request-queue, rate-limiter, semaphore	Depends on: External APIs

3. Data Flow: Input to Output

This section traces the complete path of data from external API ingestion through all processing stages to the final user-facing output. Understanding this flow is essential for identifying bottlenecks, disconnections, and orphan code paths.

3.1 Data Ingestion Layer

Data enters the system through 9 external API clients, each wrapped with the unified-cache system for TTL-based caching, rate-limit awareness, and request deduplication. The OHLCV Pipeline serves as the primary cascading data source: it tries Binance first (most reliable for major tokens), falls back to CoinGecko (broader coverage), and finally DexPaprika (best for DEX/meme tokens). DexScreener is the primary source for real-time price/volume/liquidity data. Etherscan discovers active traders for ETH tokens. Dune, Footprint, and SQD clients exist but require paid API keys that are not configured in the current environment.

Source	Data Provided	Status	Consumer
DexScreener	Price, volume, liquidity, pairs	Active	Paper trading price sync, token search, real-time data
CoinGecko	Markets, OHLCV, global data, trending	Active	OHLCV pipeline fallback, seed data, market summary
DexPaprika	Pools, swaps, OHLCV, buy/sell pressure	Active	OHLCV pipeline fallback, smart money, pressure analysis
Binance	OHLCV candles, ticker	Active	OHLCV pipeline primary source for major tokens
Etherscan	Trader discovery, transactions	Active (with key)	Smart money tracking, trader profiling

Dune Analytics	SQL queries, top traders, labels	INACTIVE (no key)	Universal data extractor
Footprint Analytics	Token prices, OHLCV, protocols	INACTIVE (no key)	Universal data extractor
SQD/Subsquid	On-chain events, transfers	INACTIVE (no key)	Historical backfill
DataIngestionPipeline	Jupiter, Solana RPC, ETH RPC	STUB ONLY	No RPC endpoints configured

3.2 Processing and Analysis Layer

Once data is ingested, it flows through multiple analysis engines that produce increasingly sophisticated outputs. The Brain Orchestrator is the central analysis hub that combines lifecycle, behavior, candlestick patterns, smart money, buy/sell pressure, operability, and deep analysis into a unified TokenAnalysis object. However, this analysis output feeds into the UI for display purposes but does NOT flow through the Strategy Decision Engine (SDE) to Capital Allocation, which is the critical gap identified in the architecture document.

Engine	Input	Output	Consumed By
OHLCV Pipeline	Raw candles from APIs	Aggregated candles by timeframe	Backtest, Pattern Detection, Regime
Token Lifecycle Engine	Price/volume/trader data	Phase classification (GENESIS to LEGACY)	Brain Orchestrator, Phase Strategy, TDE
Candlestick Pattern Engine	OHLCV candles	Pattern detections + sentiment signals	Brain Orchestrator, Pattern Compression
Behavioral Model Engine	Trader transaction history	Trader archetypes + behavioral predictions	Brain Orchestrator, Feedback Loop
Brain Orchestrator	All above + smart money + pressure	Unified TokenAnalysis	Paper Trading (partial), Deep Analysis
Regime Heuristic	Price candles + market data	Market regime (TRENDING_UP/DOWN, SIDEWAYS, etc.)	Brain Cycle (NOT connected to SDE)
Operability Score	Liquidity, volume, fees, bots	Operability score + level + max position	Paper Trading, Brain Cycle
Deep Analysis Engine	Patterns + behavior + technicals	Deep multi-depth analysis	Brain Orchestrator, UI

3.3 Decision and Execution Layer

This is the layer where the critical disconnect exists. The architecture document specifies that all strategy decisions should flow through the SDE, which would combine backtest results, Monte Carlo risk metrics, Walk-Forward efficiency, operability scores, and regime assessment into a single StrategyDecision object. This decision would then select a capital allocation method and feed into

paper trading. However, the current implementation has the Paper Trading Engine making its own inline decisions using a simple equal-split position sizing, completely bypassing both the SDE and the Capital Allocation Engine.

Module	Input	Output	Current Consumer
Strategy Decision Engine (SDE)	Backtest stats, MC risk, WF efficiency, operability, regime	StrategyDecision (state + action + quality + capital recommendation)	API routes ONLY - NOT connected to Paper Trading
Token Decision Engine (TDE)	Token lifecycle + risk + market data	Token-level decisions	Paper Trading (current scan loop)
Capital Allocation Engine	Signals, historical trades, method selection	Position size + allocation output	PTE uses calculatePositionSize() instead
Capital Strategy Manager	Learning state, mode switching	Strategy allocation decisions	Brain Cycle (NOT connected to PTE)
Paper Trading Engine	TokenAnalysis from Brain + inline logic	Paper positions + trades	API routes, UI
Kill Switch Service	Portfolio state, risk budget	Kill switch evaluations	PTE checks canOpenPosition + concentration
Monte Carlo Simulator	Trade history	Risk of ruin, P95 DD, prob of profit	API route ONLY - NOT connected to SDE
Walk-Forward Engine	Trading system + historical data	WFE, param stability, recommendation	API route ONLY - NOT connected to SDE

4. Disconnected and Orphan Modules

One of the most critical findings is the number of modules that exist in the codebase but are not connected to the main data flow pipeline. These modules produce outputs that nobody consumes, or they have the right consumers defined in the architecture document but are not wired up in the actual code. The following table documents every disconnected or partially connected module, its expected vs actual integration state, and the impact of the disconnect.

Module	Expected Integration	Actual State	Severity
Monte Carlo Simulator	Should feed riskOfRuin (veto), p95DD (allocation), probOfProfit (robustness) to SDE	Called only from /api/risk/monte-carlo API route. Output goes to UI only. Never reaches SDE or Capital Allocation.	CRITICAL
Walk-Forward Engine	Should feed aggregateWFE (robustness + veto), paramStability (stability) to SDE	Called only from /api/backtest/walk-forward API route. Output goes to UI only. Never reaches SDE.	CRITICAL

Capital Allocation Engine	Should be called by PTE for position sizing via one of 5 active methods	PTE uses its own calculatePositionSize() with equal-split instead. 16 methods exist but nobody calls them.	CRITICAL
Capital Strategy Manager	Should be absorbed into SDE per architecture doc	Called by brain-cycle-engine but NOT by paper-trading-engine or SDE	HIGH
Feedback Loop Engine	Should auto-trigger when paper trading positions close	Imported in PTE but never called automatically. Only accessible via API.	HIGH
Risk Controls Verifier	Should perform real risk controls analysis	Hardcoded responses, no real analysis performed	MEDIUM
Alert Engine	Should have escalation chain INFO->WARNING->CRITICAL->AUTO_PAUSE	Creates alerts but no escalation logic exists	MEDIUM
Strategy Evolution Engine	Evolved strategies should require SDE validation before activation	Auto-activates if minSharpeRatio + minWinRate met, bypassing SDE	HIGH
Regime Heuristic	Should feed regime to SDE for method selection + threshold adjustment	Called from brain-cycle but NOT connected to SDE	HIGH
Cross-Correlation Engine	Should feed predictive correlations to SDE	Called by brain-orchestrator and brain-analysis-pipeline, but not by SDE	MEDIUM

5. Dead Code and Redundant Modules

Several files exist in the codebase that are either entirely non-functional (stubs, architecture references, or clients with no API keys), or duplicate functionality that exists elsewhere. These add maintenance burden and confusion without providing value.

File/Module	Description	Status
trade-executor-arch.ts	Future execution architecture type definitions and placeholder class. All methods throw or return stubs.	Architecture reference only
data-ingestion.ts	JupiterClient, SolanaRpcClient, EthereumRpcClient classes. No RPC endpoints configured, no data flows.	Requires paid/complex RPC setup
sqd-client.ts	SQD/Subsquid API client. Requires API key not in .env.	Dead without API key
sqd-flipside-client.ts	Duplicate of sqd-client.ts with slightly different data source. Also requires API key.	Duplicate + no API key
dune-client.ts	Dune Analytics API client. Requires DUNE_API_KEY not in .env.	Dead without API key
footprint-client.ts	Footprint Analytics API client. Requires FOOTPRINT_API_KEY not in .env.	Dead without API key

project-system-matcher.ts	Unclear purpose vs trading-system-matcher.ts. Appears to be an older version.	Duplicate/legacy
CrossChainWallet model	Prisma model defined but NO database operations found in any source file.	Schema-only, zero usage
OperabilityScore model	Overlaps with OperabilitySnapshot model. Different granularity but similar purpose.	Duplicate model
TradingCycle model	Overlaps with BrainCycleRun model. Two separate cycle-tracking models.	Duplicate model
6 allocation methods	FIXED_FRACTIONAL, FIXED_RATIO, FIXED_AMOUNT, SCORE_BASED, RL_ALLOCATION, ADAPTIVE marked @deprecated	Deprecated but not removed

6. Database Schema Concerns

The Prisma schema contains 47 models across 1512 lines. Several structural issues were identified that affect data integrity, performance, and maintainability.

Issue	Impact	Severity	Recommendation
Zero cascading deletes	No onDelete/onUpdate directives on any FK relation. Deleting a parent with children will fail unless children are manually deleted first.	HIGH	Add onDelete: Cascade where appropriate, or ensure application code always deletes children first.
Zero native Prisma enums	All enums are String fields with comment-annotated values. No database-level validation of allowed values.	MEDIUM	Consider adding check constraints or validating in application layer.
Non-relation FK strings	AIBestStrategy.backtestId, DecisionAudit.strategyId, SystemEvolution.parentSystemId/childSystemId are plain Strings without @relation.	MEDIUM	Prisma will not enforce referential integrity for these. Invalid IDs can be stored.
SQLite concurrency	SQLite supports only one writer at a time. Concurrent API requests may serialize writes or timeout.	MEDIUM	Acceptable for single-user demo. Production would need PostgreSQL.
Nullable userId on many models	TradingSystem, BacktestRun, PaperTradingSession, etc. have nullable userId, allowing orphaned data.	LOW	Auth is disabled so all data belongs to demo user. Will matter when auth is enabled.
JSON stored as String	All complex data (signals, conditions, metadata) stored as JSON strings, not native JSON columns.	LOW	SQLite has no native JSON type. Queries into JSON fields require string parsing.

maxDailyVaR missing	RiskBudget model per architecture doc should include maxDailyVaR field, but it is absent from the schema.	HIGH	Add maxDailyVaR Float field to RiskBudget model as specified in ARCHITECTURE_FINAL.md.
Duplicate cycle models	TradingCycle and BrainCycleRun both track cycle execution with overlapping fields.	MEDIUM	Consolidate into a single model or clarify the distinct purpose of each.

7. Critical Missing Connections (vs Architecture Document)

The ARCHITECTURE_FINAL.md document specifies 13 integration points that should exist in the system. Of these, only 3 are fully implemented, 4 are partially implemented, and 6 are completely missing. The following table maps the documented requirements against the actual implementation state.

Connection	Expected Flow	Status	Priority
1. MC Simulator -> SDE	riskOfRuin -> veto, p95DD -> allocation, probOfProfit -> robustness	NOT IMPLEMENTED	P0
2. Walk-Forward -> SDE	aggregateWFE -> robustness + veto, paramStability -> stability	NOT IMPLEMENTED	P0
3. Backtest -> SDE	Base data for all scores and vetos	PARTIAL	SDE reads backtest data but does not use it in veto/score pipeline
4. Operability -> SDE	Operability score as SDE input	PARTIAL	SDE has operability in input type but it is not always populated
5. Regime -> SDE	Regime -> method selection + threshold adjustment	NOT IMPLEMENTED	P1
6. Evolution -> SDE	Evolved strategies must pass SDE validation	NOT IMPLEMENTED	Auto-evolution activates without SDE

7. SDE -> Capital Allocation	Selects method from 5 active; provides inputs for sizing	PARTIAL	SDE produces capitalRecommendation but PTE ignores it
8. SDE -> Paper Trading	Replaces token-decision-engine in scan loop	NOT IMPLEMENTED	PO
9. Paper Trading -> SDE feedback	Close position -> re-evaluate -> update audit	NOT IMPLEMENTED	PO
10. SDE -> DecisionAudit	Every call writes audit record	IMPLEMENTED	SDE creates Decision Audit on every call
11. Kill Switches -> PTE	Enforce risk controls before opening positions	IMPLEMENTED	PTE checks canOpenPosition + concentration
12. Risk Budget -> SDE	Concentration limits in capital recommendation	PARTIAL	SDE checks concentration but not max DailyVaR
13. Capital Allocation -> PTE	Sizing output feeds into execution	NOT IMPLEMENTED	PTE uses inline calculatePositionSize()

8. UI Data Flow Architecture

The frontend is a single-page application with 19 tabs, all rendered within a single page.tsx using a Zustand-driven tab system. Data reaches the UI through two channels: WebSocket real-time feeds and REST API polling via React Query. The WebSocket server (port 3010) pushes token updates, signals, brain cycle events, and alerts. When WebSocket is disconnected, the SimulationProvider falls back to REST API polling. All components are client-side rendered with dynamic imports and SSR disabled.

Data Source	Channel	Target	Behavior
-------------	---------	--------	----------

WebSocketProvider	Socket.IO (port 3010)	useCryptoStore	Feeds tokens, signals, alerts, trader stats, market summary
SimulationProvider	REST API polling	useCryptoStore	Fallback when WS offline. Fetches market summary + signals.
React Query (useQuery)	REST API per-component	Local component state	Each component fetches its own data independently
Deep Analysis Store	POST /api/deep-analysis	useDeepAnalysisStore	Separate Zustand store for deep analysis state machine

A notable observation is that many components make independent API calls for data that could be shared through the Zustand store. For example, the PortfolioView, RiskDashboard, and AllocationDashboard each call their own API endpoints independently rather than sharing a common portfolio state. This creates redundant network requests and potential data inconsistency between tabs.

9. Most-Depended-Upon Modules

The following modules are imported by the highest number of other modules. Changes to these modules have the widest blast radius and should be treated with maximum caution during any refactoring.

Module	# Consumers	Role	Consumed By
@/lib/db	30+	Prisma client singleton	ALL services and API routes that access the database
@/lib/unified-cache	10+	Centralized caching layer	All data source clients + smart-money-tracker + buy-sell-pressure
token-lifecycle-engine	12+	Token phase classification	brain-orchestrator, brain-cycle, brain-analysis, candlestick, behavioral, pattern-compression, phase-strategy, backtest-loop, feedback-loop, token-decision, deep-analysis, cross-correlation
brain-orchestrator	5+	Core analysis hub	paper-trading-engine, trade-execution-engine, brain-analysis-pipeline, cross-correlation, deep-analysis
ohlcv-pipeline	5+	OHLCV data pipeline	brain-orchestrator, brain-cycle, backtesting-engine, backtest-data-bridge, candlestick-pattern
trading-system-engine	5+	Trading system templates	trade-execution-engine, paper-trading, backtesting, backtest-data-bridge, walk-forward
dexscreener-client	7+	Primary price source	paper-trading, sync-shared, dexpaprika, real-data-loader, signal-generators, auto-evolution
operability-score	3+	Token operability assessment	paper-trading, brain-orchestrator, brain-cycle

feedback-loop-engine	4+	Trade feedback learning	paper-trading, brain-orchestrator, brain-cycle, brain-analysis-pipeline
kill-switch-service	2+	Emergency risk controls	paper-trading, capital-allocation pipeline API

10. Prisma Model Usage Analysis

Of the 47 Prisma models, some are heavily used across dozens of files while others have zero or near-zero usage. This analysis identifies which models are core to the system and which may be dead schema.

Model	Importance	Usage	Operations
Token	CRITICAL	30+ files	create, findMany, findFirst, findUnique, update, upsert, count, aggregate
TradingSystem	CRITICAL	15+ files	create, findMany, findFirst, findUnique, update, delete, count
BacktestRun	HIGH	10+ files	create, findMany, findUnique, update, delete, count
PriceCandle	HIGH	10+ files	findMany, findFirst, update, upsert, deleteMany, count, aggregate
Trader	HIGH	10+ files	create, findMany, findFirst, findUnique, update, upsert, count
BacktestOperation	MEDIUM	8 files	create, findMany, findFirst, update, deleteMany, count
Signal	MEDIUM	8 files	create, findMany, findFirst, findUnique, upsert, count
PaperTradingSession	MEDIUM	5 files	create, findFirst, findMany, findUnique, update, deleteMany
CrossChainWallet	NONE	0 files	Defined in schema but NO database operations in any source file
WalletTokenHolding	LOW	2 files	upsert only (smart-money-tracker, sqd-flipside-client)
TradingCycle	LOW	1 file	Defined but only referenced in seed/stats, unclear active use

11. Security and Authentication

Authentication is completely disabled across the entire application. The auth.ts module is a stub that returns a demo user (demo@cryptoquant.com). The getCurrentUserId() function used by some API routes returns a hardcoded demo user ID. No middleware enforces authentication on any route. Only 5 out of 104+ routes even check getCurrentUserId(). No rate limiting, CORS, or input sanitization middleware exists. While this is acceptable for a local development/demo environment, it must be addressed before any deployment. The following table summarizes the auth state across key route groups.

Route Group	Auth Level	Notes
Trading Systems	getCurrentUserId + userScope	Only route group with consistent auth checks
Alerts	getCurrentUserId	User-scoped but auth is stub
Webhooks	getCurrentUserId	User-scoped but auth is stub
Templates	getCurrentUserId + templateScope	User-scoped but auth is stub
Brain Start-All	getCurrentUserId	Single brain route with auth check
All other routes (99+)	NONE	No auth check whatsoever

12. Summary of Critical Findings

12.1 System-Stopping Issues (Must Fix Before Trading)

- The complete pipeline from SDE to Capital Allocation to Paper Trading is NOT connected. Paper Trading makes its own decisions without SDE validation, without Capital Allocation Engine sizing, and without Monte Carlo or Walk-Forward risk assessment.
- Auto-feedback loop is NOT implemented. When paper trading positions close, the system does not re-evaluate the strategy through SDE or update DecisionAudit records. The system cannot learn from its own results.
- Evolved strategies can be activated without SDE validation. The auto-evolution-loop activates strategies based on minSharpeRatio + minWinRate thresholds, bypassing the SDE veto/score/quality pipeline entirely.

12.2 High-Priority Issues

- Monte Carlo and Walk-Forward results flow to UI only, never to SDE. These validation modules produce critical risk metrics that should drive strategy decisions.
- Regime heuristic is not connected to SDE. Market regime should influence allocation method selection and threshold adjustments.
- RiskBudget is missing maxDailyVaR field specified in architecture document.
- Alert escalation chain (INFO to WARNING to CRITICAL to AUTO_PAUSE) does not exist. Alerts are created but never escalate.
- Risk Controls Verifier returns hardcoded responses instead of performing real analysis.
- 5 data source clients (Dune, Footprint, SQD x2, DataIngestionPipeline) are dead code due to missing API keys or unconfigured endpoints.

12.3 Medium-Priority Issues

- Zero cascading deletes in Prisma schema. Manual cleanup required for any deletion operation.

- 6 deprecated allocation methods still in codebase. Should be removed or clearly isolated.
- Duplicate models: TradingCycle vs BrainCycleRun, OperabilityScore vs OperabilitySnapshot.
- CrossChainWallet model has zero database operations. Dead schema.
- Non-relation FK strings in AlBestStrategy, DecisionAudit, SystemEvolution bypass Prisma referential integrity.
- TDE and SDE coexist without integration. TDE makes token-level decisions, SDE makes strategy-level decisions, but they do not share information.

12.4 Low-Priority Issues

- Authentication is completely disabled. Acceptable for demo, must be addressed for production.
- TypeScript 6.0.3 vs typescript-eslint peer dependency warnings. Non-breaking but noisy.
- Many UI components make independent API calls instead of sharing state through Zustand store.
- All enums are String fields without database-level validation. Risk of invalid data.
- SQLite is acceptable for single-user demo but will not scale to multi-user production.

END OF PHASE 1 REPORT - No modifications were made to the codebase. This document serves as the foundation for Phase 2 (Comparison with Architecture Document) and Phase 3 (Individual Module Audit).